'Assignment-02'

Amirun Nahin

ID: 23201416

Sec: 10

Course: CSE-341

Semester: Summer 2026

Date: 05/08/2026

Question-1

(a)

Two features of BIU are:

1) It reads data from port/memory and writes data into it.

2) It supports instruction queuing.

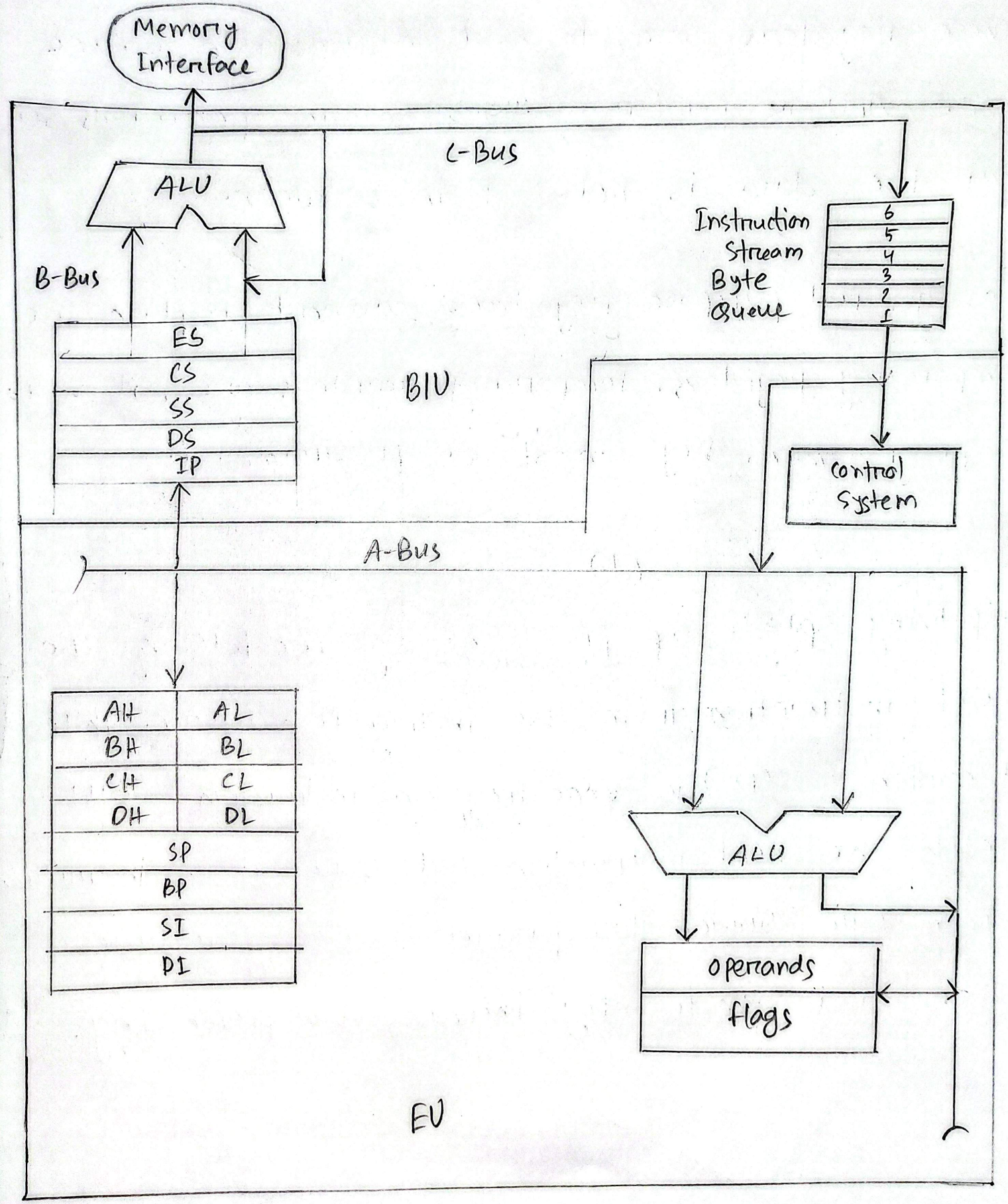
Two features of EU are:

1) It fetches instruction from BIU, decodes and executes arithmetic and logical operations using ALU.

2) It sends request to BIU for accessing external module.

(b)

The block diagram of 8086 microprocessor:



(c)

Temporary registers are internal registers that are not visible to the programmers. The purpose of these registers are to hold intermediate values while ALU is performing arithmetic operations so that the data is not lost in between.

No, general purpose registers cannot replace the temporary registers. Temporary registers are not visible to programmer. They cannot be programmed.

(d)

Pipelining speeds up processing by pre-fetching the next instruction from the instruction queue. While Execution unit (EU) is executing an instruction, BIU fetch the next instruction and stores it in Instruction Stream Byte Queue. This parallel processing reduces delay and significantly increases the processing.

(e)

Pipelining fails during branch instructions such as JMP or CALL. During these type of instructions execution, the flow abruptly changes. Meaning the next instruction to be executed is not the one currently stored in instruction queue. Microprocessor then discard the instructions in the queue and fetch the new instructions.

Question-2

(a)

8086 microprocessor has a flag register (16 bits) where 7 bits are undefined, 3bits for control flags and 6 bits are for status flags.

Control flag are 3 bits that control the operation of the cpu.

1) Direction flag (DF): Flag is set(1), when processing is done forward.

2) Interrupt flag (IF): set(1) when cpu reacts to interrupts from external devices.

3) Trap flag (TF): set(1) when single step debugging mode is ON

Status flags are 6 bits that holds the result after the last arithmetic operation.

1) Overflow flag (OF): set (1) when there is a signed overflow.

2) Sign flag (SF): set(1) when result is negative.

3) Zero flag (ZF): set(1) when result is zero.

1) Auxilary flag (AF): set(1) when there is an unsigned overflow for lower nibble.

5) parity flag (PF): set(1) when there is an even number of 1 in result's lower byte.

6) Carry flag (CF): set(1) when there is an unsigned overflow.

These status flags are used in error detection. For example, Parity flag and overflow flag can detect logical errors in data. If transmitted data is expected with even parity but programmer gets odd parity (PF=0), the programmer can detect an error. Similarly if OF=1, the programmer knows the result has overflowed the range.

(b)

For overflow flag to be set (OF=1), two cases are:

1) Carry into MSB = 1

Carry out of MSB = 0

$$\begin{array}{rcl} & & {\scriptstyle 1} \\ +64 & = & 0100\ 0000 \\ +64 & = & 0100\ 0000 \\ \hline & & 1000\ 0000 \end{array}$$

Here two positive numbers are added. However, the answer is a negative number $(-128)_{10}$. So, a signed overflow occurred.

2) Carry into MSB = 0

Carry out of MSB = 1

$$\begin{array}{rcl} & & {\scriptstyle 1} \\ -128 & = & 1000\ 0000 \\ -128 & = & 1000\ 0000 \\ \hline & & 10000\ 0000 \end{array}$$

Here, two negative numbers are added. However, the result is 0. So, a sign overflow has occurred.

(c) (i)

Given values,

FFCD and FFXY

for PF=1, AF=0, maximum values of X and Y are F and 1.

FFCD	=	1111	1111	1100	1101
FFF1	=	1111	1111	1111	0001
		1111	1111	1011	1110

So, X = F and Y = 1. Other values of status flags are:

OF = 0	AF = 0
SF = 1	CF = 1
ZF = 0	PF = 1

(ii)

for PF=0, AF=0, minimum values of X and Y are 0 and 0.

FFCD	=	1111	1111	1100	1101
FF00	=	1111	1111	0000	0000
		11111	1110	1100	1101

So, X=0, Y=0. Other status flags' values are:

OF = 0	AF = 0
SF = 1	CF = 1
ZF = 0	PF = 0

(iii)

for AF=1, the maximum value of X = F and minimum value of Y = 3.

FFCD	=	1111	1111	1100	1101
FFF3	=	1111	1111	1111	0011
		1 1111	1111	1100	0000

So, X = F, Y = 3. Other values of status flags are:

OF = 0 AF = 1

SF = 1 PF = 1

ZF = 0 CF = 1

Question-3

(a)(i)

Given instruction,

MOV DH, [SI + 0078h]

MOV opcode = 100010

D = 1 since destination operand is register

W = 0 since 8 bit data

MOD = 01

REG = 110

R/M = 100

byte 1: | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | = 8Ah

byte 2: | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | = 74h

byte 3: | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | = 78h

So, machine code 8A 74 78h

(ii)

Given instruction,

MOV [BP+SI+1971h], AX

MOV opcode = 1000 10

D = 0 since source operand is register

W = 1 since 16 bit data

MOD = 10

REG = 000

R/M = 010

byte 1: | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | = 89h

byte 2: | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | = 82h

byte 3: | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | = 71h

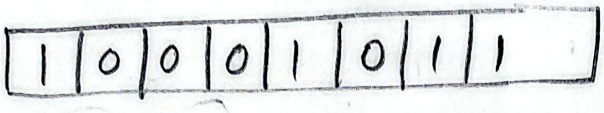
byte 4: | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | = 19h

So, machine code 89 82 71 19 h

(b)(i)

given machine code,

8BC1 h

byte 1:  = 8Bh

opcode = 100010 so, MOV operation

D = 1, so destination is the register

W = 1, so 16 bits of data

byte 2:  = C1h

MOD = 11 so, both source and destination are register

REG = 000, so destination register is AX

R/M = 001, so source register is CX.

so, the final instruction is MOV AX, CX

Now for alternative machine code, we can take,

D=0. Then REG becomes 001 and R/M is 000.

So, MOV = 100010

D = 0

W = 1

MOD = 11

REG = 001

R/M = 000

Byte 1: | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | = 89h

Byte 2: | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | = C8h

So, alternative machine code = 89 C8h

(ii)

Given machine code,

89 91 24 20 h

Byte 1: | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | = 89h

Opcode = 100010 so MOV operation

D = 0, so source is the register

W = 1, so 16 bits of data

Byte 2: | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | = 91h

MOD = 10. so memory with 16 bits dispacement

REG = 010, so source register is DX

R/M = 001 so, [BX + DI] + d16

Byte 3: | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | = 24h

Byte 4: | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | = 20h

so, displacement is 2024h.

The final instruction: MOV [BX + DI + 2024h], DX

Question-4

(a)

1) MOV AX, 0005H → Immediate addressing
2) MOV [BX], AX → Register Indirect addressing
3) MOV [2222H], 0F9H → Direct addressing
4) MOV CX, [BX + 1222H] → Register - relative addressing
5) MOV DX, 0080H → Immediate addressing
6) IN AL, DX → I/O port addressing
7) OUT DX, AX → I/O port addressing
8) PUSH AX → Addressing stack in memory
9) POP CX → addressing stack in memory

10) MOV DX, CX → Register direct addressing

11) MOV [DI], DX → Register Indirect addressing

12) MOV BX, Bx[6000H] SI → Base-relative-plus-index addressing

13) HLT → Implied addressing

14) JMP [SI+0778H] → Register relative addressing

15) ADD AX, 0003H → Immediate addressing

16) RET → Implied addressing

(b)

We know,

PA = segment no. × 10h + offset

1. MOV [BX], AX

PA = (615E × 10h) + 1000h

= 625E0h

Data Written: 625E0h = 05 } AX register's value
625E1h = 00 } is loaded in the previous instruction

2. MOV [2222H], 0F9H

PA = (615E × 10h) + 2222h

= 63802h

Data written:

63802h = F9

3. MOV CX, [BX+1222H]

PA = (615E x 10h) + 1000 + 1222

= 63802h

Data written:

CL = F9

CH = cannot be determined. Since we don't know the value in the 63803h memory location.

4. PUSH AX

PA = (4021 x 10h) + F57Eh

= 4F78Eh

Data written:

4F78Dh = 00

4F78Ch = 0A

5. CALL FOO

PA = (4021 x 10h) + F57C

= 4F78Ch

Data written:

4F78Bh = Cannot be determined
4F78Ah = Cannot be determined

} Since we don't know the IP register value.

6. RET

PA = (4021 x 10h) + F57A

= 4F78A

Data written:

4F78Ah = Cannot be determined
4F78Bh = Cannot be determined

} since we don't know the IP value

7. POP CX

PA = (4021 x 10h) + F57Ch

= 4F78Ch

Data written:

CL = 0A
CH = 00

8. MOV [DI], DX

PA = (E74 x 10h) + 7888h

= 15FC8h

Data Written:

15FC8h = 0A
15FC9h = 00
} DX contains CX values in the previous instruction

9. MOV BX, BX[6000H]SI

PA = (615E X10h) + 1000h + 888h + 6000h

= 68E68h

Data written:

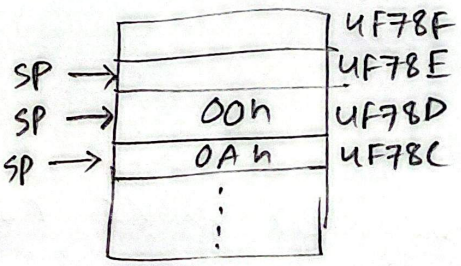
BL = Cannot be determined
BH = Cannot be determined
} since we don't know the value in 68E68h and 68E69h memory locations

(c)

Initial SP= F57Eh, PA= 4F78Eh

1) PUSH AX.

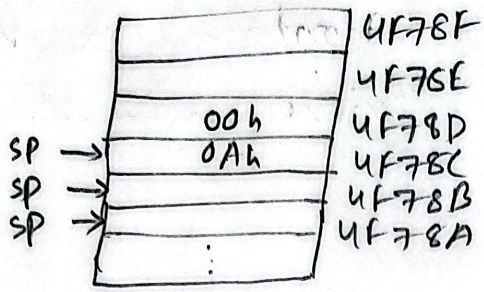
SP= F57E



new SP = 4F57Ch. AX had 000Ah data from previous instructions.

2) CALL FOO

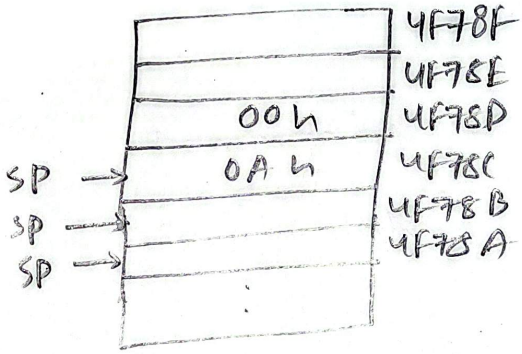
SP= F57Ch



new SP= F57A. Written data cannot be determined.

3) RET

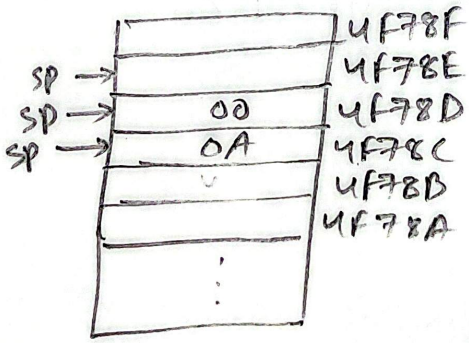
SP= F57Ah



So, new SP= F57Ch. Written data cannot be determined.

4) POP CX

SP = F57Ch



So, new SP= F57Eh. CX register value now is 000AH.

(d)

Given JMP instruction,

JMP [SI+0778H]

So, PA = (615Eh x 10h) + 0778h + 888h

= 625E0h

from the instructions done in previous questions, we had the value stored in 625E0h = 05, 625E1h = 00.

So, IP = 0005h

Now, jump address,

PA = (2010x 10h) + 0005h

= 20105h

So, JMP [SI+ 07778h] would jump to 20105h if executed.

(e)

Before the instructions run, value in AX, BX, CX, DX registers are:

AX = unknown CX = unknown

BX = 1000h DX = unknown

After the execution, done in previous answers.

AX = 000Dh

BX = Cannot Be Determined

CX = 000Ah

DX = 000Ah